

실패경로 4문항 전수 점검 보고서

30개 프로젝트 외부 의존 실패 처리 감사

점검일 2026년 7월 28일

Opus 5 오케스트레이션 · Explore 서브에이전트 8기 병렬

Gagahoho, Inc.

실패경로 4문항 전수 점검

외부 의존이 실패할 때 시스템이 사용자에게 무엇을 보여주는가 — 30개 프로젝트 감사

1. 한 장 요약

30개 프로젝트를 외부 호출 보유 여부로 필터링해 32개가 해당, 이 중 금전 흐름·호출량 기준 상위 9개를 정밀 점검했다.

정밀 점검 9 개 프로젝트	P0 결함 7 건	P1 결함 9 건	즉시 수정 4 건 완료·검증
----------------------	-----------------	-----------------	-----------------------

핵심 발견 — 결함이 특정 프로젝트에 몰려 있지 않고 9개 중 8개에서 같은 유형이 반복됐다. 개별 실수가 아니라 공통 코딩 습관의 문제다.

2. 반복된 결함 패턴 3가지

패턴 A — "실패를 성공으로 위장하는 mock" (7개 프로젝트)

외부 호출이 실패하면 하드코딩된 값을 반환하고, 호출자는 그것이 실패인지 알 수 없다.

프로젝트	위장된 값	결과
LandCheck	mock_land 500m ² × 350,000원	모든 필지가 동일한 1억 7,500만원 참고가액
CEO	mock_result level:4	허구의 "CEO 네트워킹 식대 50만원" 긴급 알림 발송
SocialDoctors	Math.random() 도달 500~5,500	조작 수치가 성과 데이터로 표시
OmniVibePro	_generate_fallback 템플릿	"기본 전략/일반/Brand" 더미를 200 OK로 반환
InsureGraph	rand(-1.0..1.0) 1536차원	랜덤 벡터를 임베딩으로 저장 (죽은 코드)
ExamsGraph	concept_gap_prob = 0.5	하드코딩 0.5가 가중치 80%로 학습 진단 반영
CPOFlow2	IN_PRICE.to_f → 0.0	단가 결측이 AED 0.00으로 카탈로그 표시

공통점 — mock 값이 구체적이다. nil이나 빈 값이 아니라 그럴듯한 숫자·문장이어서 진짜와 구별되지 않는다.

패턴 B — "출처를 알려주고도 버리는 UI" (3개 프로젝트)

API는 source: 'mock' 같은 마커를 보내는데 화면이 그 필드를 참조하지 않는다.

- SocialDoctors — InsightsResponse에 source가 있으나 프론트 InsightsData 타입에 애초에 선언조차 없어 값이 버려짐
- CEO — Disclaimer 모듈이 존재하나 attach / wrap 호출부가 grep 0건 (죽은 코드)
- Townin — forecast는 fallback: true를 잘 표기하는데, AEO 리라이트는 source 필드 자체가 없음

시사점 — 백엔드가 정직해도 프론트가 버리면 사용자에게겐 거짓말이다. 양쪽을 함께 봐야 한다.

패턴 C — "실패해도 돈은 빠진다" (3개 프로젝트)

프로젝트	결함	상태
LandCheck	차감 → 분석 순서, 트랜잭션 없음	수정 완료
Townin	AEO 유료 리라이트 실패해도 크레딧 영구 차감 (status 미필터)	미처리 P0
Townin	T0SS_SECRET_KEY 미설정 시 결제 검증 없이 자동 승인	미처리 P0

Townin의 두 번째 건은 방향이 반대다 — 돈을 안 받고 크레딧을 주는 구조로, 운영에서 환경변수가 비면 즉시 손실이 발생한다.

3. 프로젝트별 결과

#	프로젝트	1. 가짜채우기	2. 무응실패	3. 사용자인지	4. 금전정합	최고 심각도
1	LandCheck	위반→수정	위반→수정	위반→수정	위반→수정	해소
2	CEO	위반 (9개 서비스)	부분위반	위반	해당없음	P0
3	Townin Graph	위반	P2	부분위반	위반	P0
4	OmniVibePro	위반	없음	위반	해당없음	P0
5	ExamsGraph	위반	부분위반	위반	없음	P0
6	infraGrid	위반	없음	부분위반	해당없음	P0
7	SocialDoctors	위반	P2	위반→수정	없음	해소
8	CPOFlow2	위반	없음	부분위반	해당없음	P1

9	InsureGraph	위반(죽은코드)	없음	없음	해당없음	P1
---	-------------	----------	----	----	------	----

금전 정합은 대체로 견고했다. SocialDoctors의 Toss 결제·커미션은 트랜잭션 원자성과 검증 순서(외부 승인 → DB 반영)가 지켜져 있었고, ExamsGraph의 환불도 즉시 완료가 아닌 refund_requested 상태 전이로 올바르게 설계돼 있었다. OmniVibePro의 quota는 **응답 성공 시에만 차감**한다. 결제는 신경 써서 만들었다는 뜻이다.

InsureGraph는 로깅·사용자 인지 2문항 모두 "위반 없음" — 유일한 결함이 호출되지 않는 죽은 코드였다. OCR 실패를 ocr_status: failed + 에러 메시지로 화면에 그대로 노출하는 구조가 모범적이다.

infraGrid의 결함은 성격이 다르다. 결측 텔레메트리를 to_f 로 0 변환해 그래프에 그리면, 센서 미보고 구간이 "온도 급락"처럼 보인다. 인프라 모니터링에서 이런 장애 오판으로 직결된다.

4. 즉시 수정 완료 (4건)

이슈	프로젝트	수정 내용	검증 방법
ISS-028	LandCheck	mock 3종 제거, 참고가액 미산출 시 nil, credits 동적 생성, HTTP 실패 로깅(키마스킹)	Rails runner 렌더 3케이스 전부 PASS
ISS-029	LandCheck	분석 선행 → 트랜잭션 결합, 전체 실패 시 무과금 중단	실 Wallet 롤백 실측 PASS (2,900원 복구)
ISS-158	SocialDoctors	InsightsData 에 source 필드 추가, mock이면 수치 미표시 + "미연동" 안내	tsc --noEmit 에러 0
ISS-333	CEO	mock_result 제거 → ClientUnavailable 예외 → default_result (level 3)	Rails runner 실행, level=3 알림 미발송 확인

검증은 전부 직접 실행했다. 문법 검사만으로 완료 처리한 건은 없으며, LandCheck 검증 과정에서 실제로 결함 2건(총액 "0원" 표시, 테스트 판정 기준 오류)을 추가 발견해 수정했다.

5. 미처리 PO (6건) — 우선순위 순

순위	프로젝트	결함	왜 PO인가
1	Townin	AEO 유료 리라이트 실패해도 크레딧 차감 (aeo.service.ts:371)	사용자 실제 금전 손실
2	Townin	TOSS_SECRET_KEY 미설정 시 결제 무검증 승인	환경변수 누락 시 회사 손실
3	ExamsGraph	자동제출 채점 실패해도 "완료" + stale/nil 점수 표시	시험 결과 오인, nil >= threshold 크래시 가능
4	OmniVibePro	전략 생성 LLM 이중 실패 시 더미 템플릿을 200 OK로 반환	생성물이 곧 상품
5	CEO	9개 AI 서비스 전반의 mock 판정값 (절세액·등급·"pass")	세무 판단 근거 오염
6	infraGrid	텔레메트리 결측 → 0 → 그래프 급락	장애 오판

Townin 2건이 최우선인 이유는 유일하게 실제 금전이 오가는 미처리 결함이기 때문이다. 나머지는 거짓 정보 문제지만, 이 둘은 돈이 잘못 움직인다.

6. 규칙 등록 — 재발 방지

전역 `~/claude/CLAUDE.md` 에 "실패경로 4문항" 45줄 섹션을 등록했다 (2026-07-28).

#	질문	위반 신호
1	가짜 채우기 — 실패 시 하드코딩·mock으로 채우는가?	<code>foo_api</code> <code>mock_foo</code> / 기본값 상수 / 평균치 대입
2	무음 실패 — 실패가 로그에 남는가?	<code>rescue => nil</code> / <code>catch {}</code> / <code>except: pass</code> 후 로깅 없음
3	사용자 인지 — 실패를 사용자가 아는가?	출처·근거 표기가 실패 여부와 무관하게 고정
4	금전 정합 — 실패해도 돈·크레딧이 빠지는가?	차감이 작업보다 선행 / 차감·산출이 같은 트랜잭션 아님

판정 기준 — "모르는 것을 아는 척하는 값을 내보내면 안 된다."

`nil` · "조회 불가"는 정직한 출력이고, 평균값·기본값·mock은 거짓 정보다. 특히 그 값이 금액·점수·판정으로 이어지면 결함이 아니라 책임 문제가 된다.

순서 고정 — ① 폴백 정책 확정 → ② 워터폴·캐시로 실패 확률 축소.

폴백을 정하지 않고 공급사를 늘리면 전부 실패했을 때 여전히 가짜 값이 나온다. 비용만 늘고 문제는 그대로다.

이제 어느 프로젝트에서든 외부 API 코드를 건드리면 이 4문항이 컨텍스트에 자동 로드된다.

7. 이 점검이 시작된 경위

대표님이 요청한 것은 "LandCheck 워터폴 적용 시나리오"였다. 워터폴 설계에는 반드시 정해야 할 항목이 하나 있다 — "전부 실패하면 어떻게 되는가", 즉 폭포의 마지막 칸이다.

그 자리를 확인하러 코드를 열었더니 이미 채워져 있었다. `mock_land` 로.

워터폴이라는 개념이 결함을 탐지한 것이 아니라, 워터폴을 설계하려면 반드시 봐야 하는 위치에 결함이 있었다. 그 질문을 30개 프로젝트로 확장한 것이 이 점검이다.

8. 권고

즉시 (첫 유료 고객 전)

- Townin P0 2건 — 금전이 직접 걸린 유일한 미처리 건
- ExamsGraph 채점 실패 처리 — `nil >= threshold` 크래시 가능성

다음 (고객 확보 후)

- CEO · OmniVibePro · infraGrid의 mock 마커 추가
- 워터폴 · 캐시 최적화 (LandCheck 시나리오 A~D)

하지 말 것

유료 고객 0명인 프로젝트의 실패경로를 전부 정비하는 것은 회피 행동이 될 수 있다. 아무도 안 쓰는 시스템의 정직성을 높이는 일에는 상한이 있다. 이번 점검의 가치는 결함 목록이 아니라 규칙 등록에 있다 — 앞으로 만드는 코드가 같은 실수를 반복하지 않는다는 것.

LandCheck 4건과 Townin 2건은 예외다. 전자는 유료 리포트가 실제로 발행되는 경로이고, 후자는 지금도 돈이 잘못 움직일 수 있다.